KernelLab - Simulador de Gerenciamento de CPU, Memória e Dispositivos em Sistemas Multitarefa

Relatório técnico - Parte Teórica

Disciplina: Sistemas Operacionais - UniFECAF

Autor(a): Hugo Vieira

Repositório: https://github.com/hugofelipe22/KernelLab

Vídeo: https://drive.google.com/file/d/1LoJspIvDFljWNC1lrjUZcHG6iQ36kUoS/view?usp=sharing

1. Contextualização do problema

A MedControl Systems opera uma plataforma de monitoramento de equipamentos médicos conectados. Em um único servidor, com **uma CPU**, executam simultaneamente seis processos: monitoramento cardíaco (P1), transmissão de telemetria (P2), geração de relatórios (P3), emissão de alarme crítico (P4), backup (P5) e atualização de dados (P6). Juntos demandam 41 unidades de CPU e 910 MB de memória, e competem por disco, rede e áudio.

Os sintomas observados pela empresa correspondem diretamente a tópicos da disciplina:

Sintoma	Conceito de SO envolvido	Onde é tratado no KernelLab
Atraso no processamento de alarmes	Política de escalonamento; prioridade; efeito comboio	Etapas 1 e 2 (espera do P4 em cada algoritmo)
Aumento do tempo de resposta	Tempo de resposta; tamanho do quantum; preempção	Etapa 1 (métrica de resposta)
Disputa pela CPU	Multiprogramação; troca de contexto	Etapa 1 (trocas de contexto)
Elevado número de faltas de página	Memória virtual; política de substituição	Etapa 3 (FIFO, LRU, Optimal)
Processos de baixa prioridade esperando demais	Starvation / inanição; justiça	Etapas 1 e 2 (atraso excessivo, índice de Jain)
Concorrência no acesso a dispositivos	Condição de corrida; exclusão mútua; mutex e semáforos	Etapa 4 (fila FIFO + mutex)
Processos não autorizados usando recursos críticos	Proteção; matriz de controle de acesso; menor privilégio	Etapa 4 (matriz de acesso)

O objetivo do KernelLab não é ser um sistema operacional, e sim reproduzir as *decisões* que um SO toma (quem usa a CPU, qual página sai da memória, quem acessa o dispositivo) para que o impacto de cada política possa ser medido e comparado sobre os mesmos dados.

2. Estados e ciclo de vida dos processos

Um processo é um programa em execução, com seu contexto (registradores, contador de programa, espaço de endereços). Em um sistema multiprogramado, cada processo percorre os estados:

Estado	Significado	Exemplo na MedControl
Novo	Processo sendo criado	P5 (backup) é disparado no instante 5
Pronto	Pode executar, aguarda a CPU	P3, P4, P5 e P6 aguardando enquanto P1 usa a CPU
Executando	Está de fato usando a CPU	P4 emitindo o alarme
Bloqueado (espera)	Aguarda evento externo, p. ex. dispositivo de E/S	P5 aguardando o disco ocupado por P3
Terminado	Concluiu a execução	P1 após consumir seus 8 de burst

As transições relevantes são: **novo -> pronto** (admissão), **pronto -> executando** (despacho pelo escalonador), **executando -> pronto** (preempção por fim de quantum ou por chegada de processo mais prioritário), **executando -> bloqueado** (pedido de E/S ou de recurso ocupado), **bloqueado -> pronto** (recurso liberado) e **executando -> terminado**. O escalonador de CPU atua nas transições envolvendo o estado *pronto*; o KernelLab modela cada processo apenas com tempo de chegada e burst de CPU, de modo que a transição para *bloqueado* aparece na Etapa 4 (fila de dispositivo), não na Etapa 1.

3. Escalonamento de CPU

O escalonador decide, a cada instante, qual processo pronto recebe a CPU. A **troca de contexto** é o custo de salvar o estado de um processo e restaurar o de outro; no simulador é contada cada vez que a CPU passa de um processo para outro diferente (o primeiro despacho não conta e o custo em tempo da troca é considerado nulo).

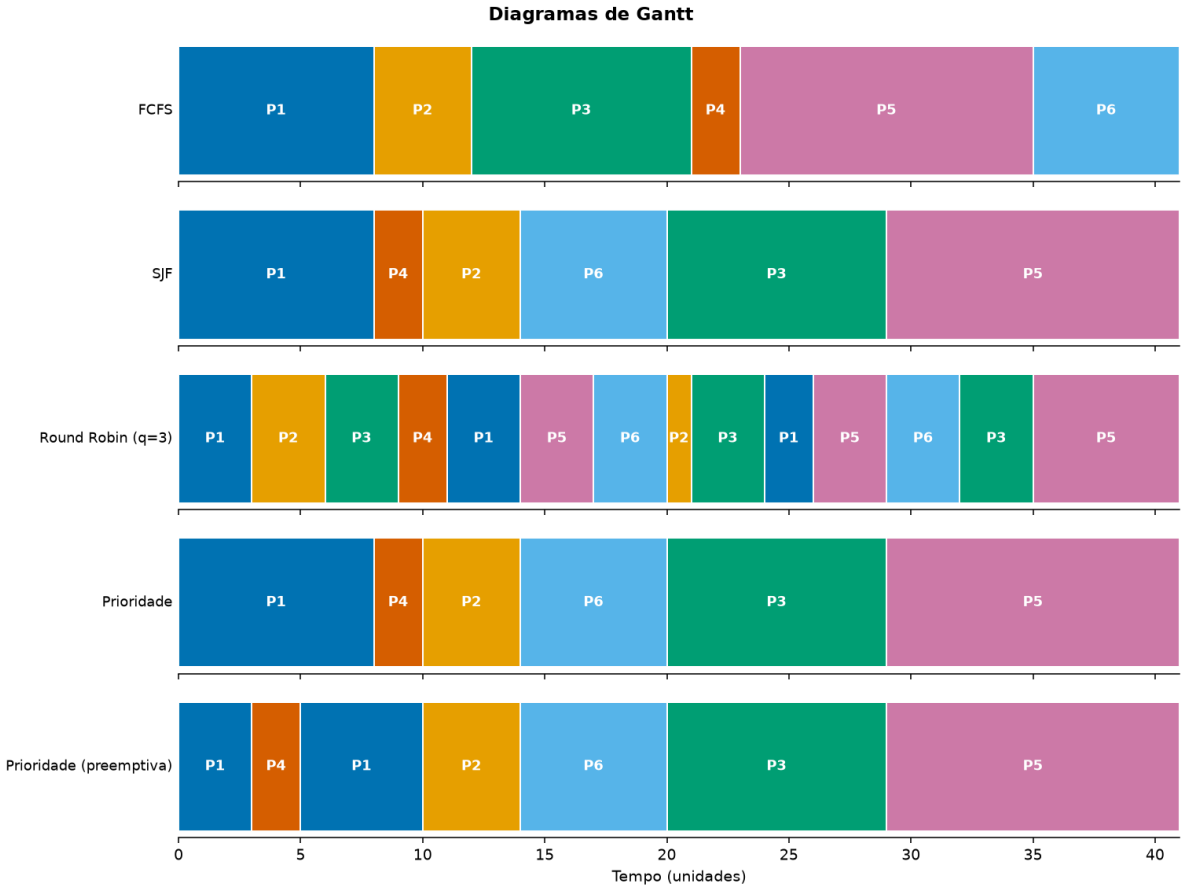
3.1 Algoritmos implementados

- **FCFS** (First Come, First Served): não preemptivo; atende na ordem de chegada. Simples e sem starvation, mas sofre com o *efeito comboio*: processos curtos esperam atrás de processos longos.
- **SJF** (Shortest Job First, não preemptivo): entre os prontos, escolhe o de menor burst. Minimiza a espera média, porém exige conhecer o burst e pode adiar indefinidamente processos longos.
- **Round Robin** (quantum = 3): preemptivo; cada processo usa a CPU por no máximo um quantum e volta ao fim da fila. Dá boa resposta e justiça, ao custo de mais trocas de contexto e maior retorno para processos longos.
- **Prioridade** (menor número = maior prioridade): escolhe o pronto mais prioritário. Expressa criticidade, mas pode causar starvation dos de baixa prioridade. Implementada na forma não preemptiva (padrão do enunciado) e, como **variante extra**, na forma preemptiva, em que a chegada de um processo mais prioritário interrompe a CPU.

3.2 Métricas

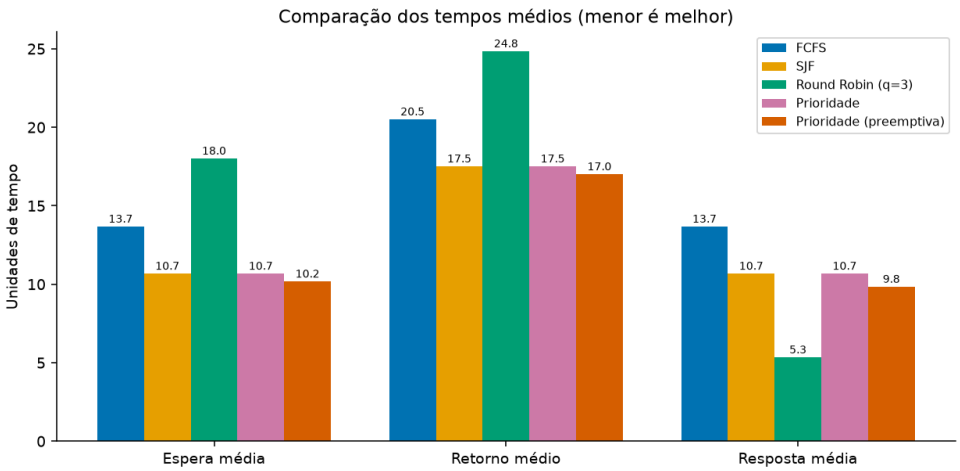
- **Tempo de retorno** = instante de término - chegada.
- **Tempo de espera** = retorno - burst (tempo parado na fila de prontos).
- **Tempo de resposta** = primeira vez na CPU - chegada.
- **Atraso excessivo** (critério do simulador): espera >= 2 x o próprio burst.
- **Risco de starvation** (critério do simulador): o processo foi ultrapassado por algum processo que chegou depois dele *e* sua espera é >= 2 x a espera média do algoritmo. Em uma simulação finita não é possível provar inanição; o critério sinaliza o *risco*.
- **Justiça**: índice de Jain sobre burst/retorno de cada processo (fração do tempo no sistema em que ele recebeu CPU); varia de 1/n a 1.

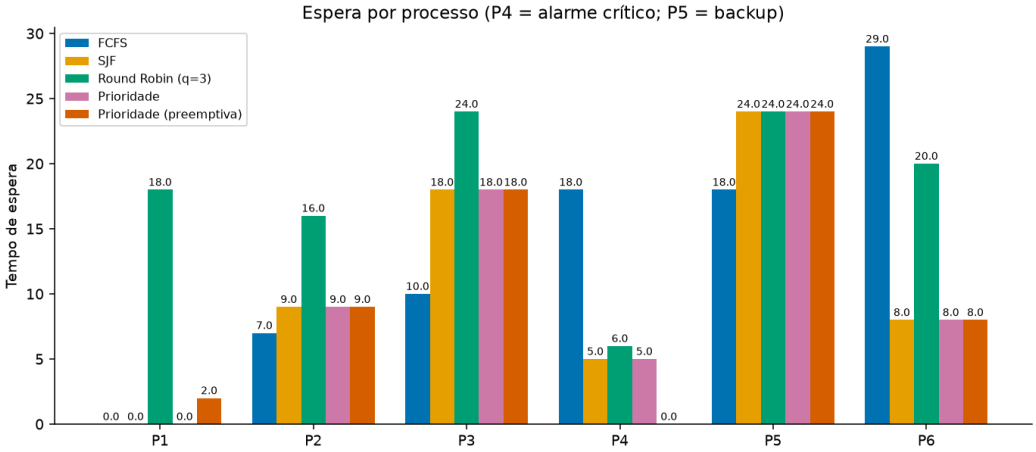
3.3 Resultados sobre os processos do enunciado



Algoritmo	Espera média	Retorno médio	Resposta média	Trocas de contexto	Jain	Espera do alarme (P4)	Em risco de starvation
FCFS	13,67	20,50	13,67	5	0,674	18	nenhum
SJF	10,67	17,50	10,67	5	0,761	5	P5
Round Robin (q=3)	18,00	24,83	5,33	13	0,972	6	nenhum
Prioridade	10,67	17,50	10,67	5	0,761	5	P5
Prioridade (preemptiva)	10,17	17,00	9,83	6	0,799	0	P5

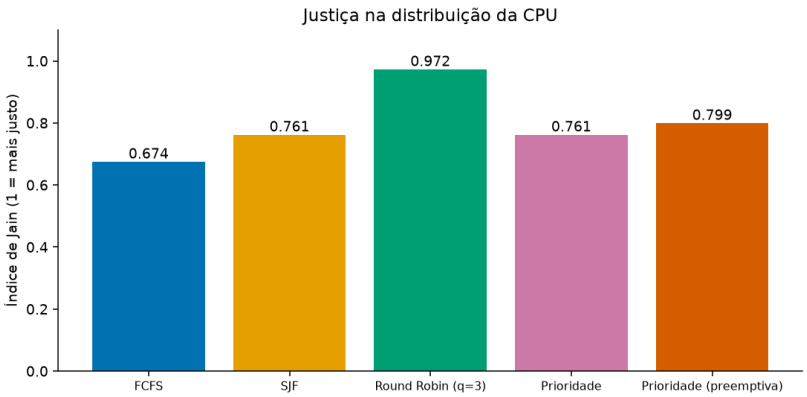
Valores em unidades de tempo. A linha 'Prioridade (preemptiva)' é a variante extra.

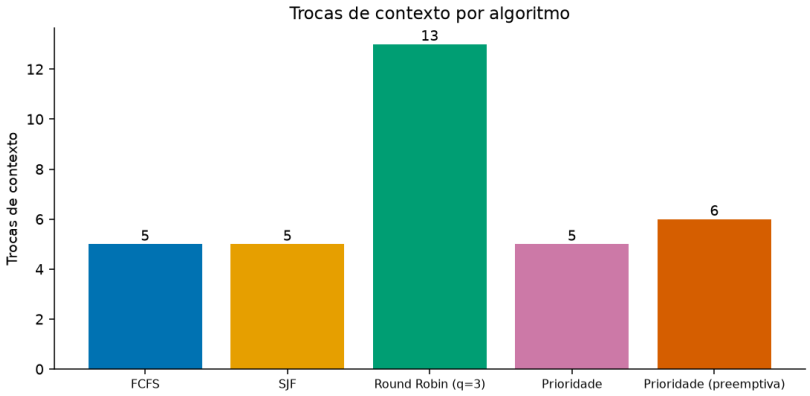




3.4 Respostas da comparação (Etapa 2)

Pergunta	Resposta (entre os 4 algoritmos obrigatórios)	Justificativa
Menor tempo médio de espera	SJF e Prioridade (empate, 10,67)	Ambos executam P1, P4, P2, P6, P3, P5. Nesta carga a ordem por prioridade coincide com a ordem por burst; por isso os resultados são idênticos.
Melhor tempo de resposta	Round Robin (5,33)	O quantum faz todo processo ver a CPU logo após chegar (pior resposta individual: 11, de P6).
Alarme (P4) tratado com mais eficiência	SJF e Prioridade (espera 5, retorno 7); RR: espera 6; FCFS: espera 18	Sem preempção, o P4 precisa esperar P1 terminar (instante 8). Só a variante preemptiva atende o alarme imediatamente: espera 0.
Maior risco de starvation	SJF e Prioridade (P5: espera 24, ultrapassado por P6)	Processos longos e/ou de baixa prioridade são sempre preteridos. FCFS e RR não ultrapassam ninguém; no FCFS o dano é o comboio (P4 espera 18).
CPU distribuída de forma mais justa	Round Robin (Jain 0,972)	Todos recebem a CPU em fatias iguais e nenhum é ultrapassado; o preço é 13 trocas de contexto e a pior espera média (18,00).





3.5 Qual é o mais adequado à MedControl?

Nenhuma métrica isolada decide. O processo P4 (alarme crítico) tem prioridade 0 e burst de apenas 2: o que importa para ele é *não esperar*. O Round Robin é o mais justo e o de melhor resposta média, mas faz o alarme esperar 6 unidades e tem a maior espera média (18,00), além de 13 trocas de contexto. O FCFS é o pior para o alarme (espera 18): ele fica atrás do relatório (P3), um processo sem urgência. O SJF reduz bem a espera média, mas só conhece duração, não criticidade: um alarme longo seria penalizado.

Recomendação: escalonamento por **prioridade**. Entre os quatro algoritmos obrigatórios, a Prioridade empata com o SJF em desempenho (10,67 de espera média) e é a única que expressa criticidade explicitamente. Com **preempção** (variante extra), o alarme é atendido sem espera (P4: 0, retorno 2), a espera média cai para 10,17 e o custo é uma única troca de contexto a mais (6 contra 5). Como a prioridade sozinha mantém o risco de starvation do backup (P5 espera 24), a política deve ser acompanhada de um mecanismo contra inanição (ver seção 8).

4. Memória virtual e substituição de páginas

Na memória virtual, o espaço de endereços de cada processo é dividido em **páginas**, mapeadas para **frames** da memória física. Nem todas as páginas precisam estar na RAM: ao referenciar uma página ausente ocorre uma **falta de página (page fault)**, o SO a busca no disco e, se não há frame livre, precisa escolher uma **vítima**. Como o acesso ao disco é ordens de grandeza mais lento que o da RAM, o número de faltas determina diretamente a latência dos processos. Na MedControl, os seis processos somam 910 MB; se a memória física não comporta tudo, páginas do alarme e do monitoramento podem ser expulsas por processos pesados como o backup (300 MB), atrasando respostas.

- **FIFO**: expulsa a página que está há mais tempo na memória. Fácil de implementar, mas ignora o uso: pode expulsar uma página muito utilizada e sofre da *anomalia de Belady* (mais frames podem gerar mais faltas).
- **LRU** (Least Recently Used): expulsa a página usada há mais tempo, aproximando o futuro pelo passado recente (princípio da localidade).
- **Optimal**: expulsa a página cujo próximo uso é o mais distante. É irrealizável na prática (exige conhecer o futuro) e serve de limite inferior para comparação.

Sequência: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, 2, 1, 5, 3, 2, com 4 frames.

Algoritmo	Page hits	Page faults	Taxa de faltas
FIFO	5	12	70,6%
LRU	7	10	58,8%
Optimal	10	7	41,2%

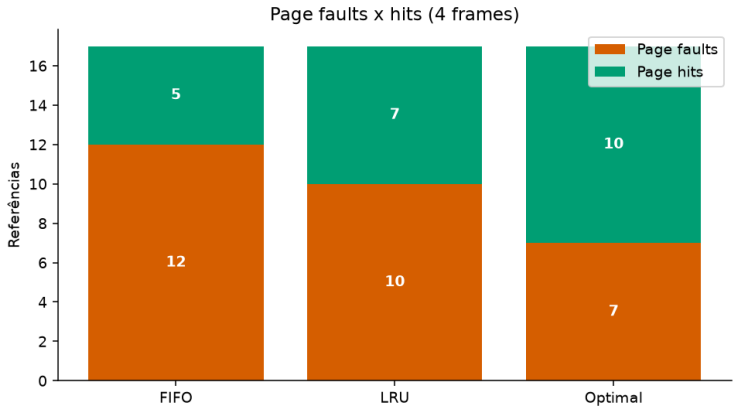
Estado dos frames do LRU após cada referência (os de FIFO e Optimal estão na saída do simulador):

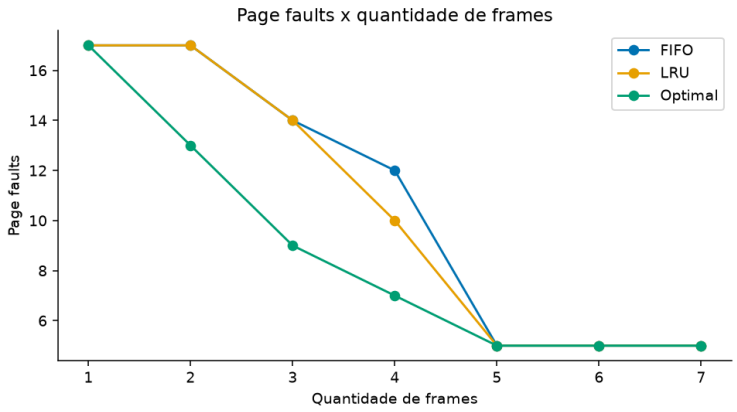
Ref	F1	F2	F3	F4	Resultado
1	1	-	-	-	FALTA
2	1	2	-	-	FALTA
3	1	2	3	-	FALTA
4	1	2	3	4	FALTA
1	1	2	3	4	hit
2	1	2	3	4	hit
5	1	2	5	4	FALTA (sai 3)
1	1	2	5	4	hit
2	1	2	5	4	hit
3	1	2	5	3	FALTA (sai 4)
4	1	2	4	3	FALTA (sai 5)
5	5	2	4	3	FALTA (sai 1)
2	5	2	4	3	hit
1	5	2	4	1	FALTA (sai 3)
5	5	2	4	1	hit
3	5	2	3	1	FALTA (sai 4)
2	5	2	3	1	hit

Melhor resultado e justificativa. O Optimal faz 7 faltas, o LRU 10 e o FIFO 12. Entre os implementáveis, o LRU é o melhor. A diferença nasce da escolha da vítima: ao chegar a referência 5 (posição 7), o FIFO expulsa a página 1, a mais antiga, mas ela é referenciada logo em seguida, provocando faltas em cascata (1, 2, 3, 4 sendo trocadas sucessivamente). O LRU também sofre nessa região, porém preserva as páginas recentemente usadas (1 e 2) e, assim, acerta mais referências (7 hits contra 5). O Optimal, ao enxergar o futuro, expulsa as páginas (4 e 3) cujo próximo uso está mais longe, chegando a 10 hits.

Efeito do número de frames. A varredura abaixo mostra que mais frames reduzem as faltas, e que a partir de 5 frames só restam as 5 faltas compulsórias (uma por página distinta). Para o FIFO com as 12 primeiras referências, o simulador reproduz a anomalia de Belady: 9 faltas com 3 frames e 10 com 4.

Frames	FIFO	LRU	Optimal
1	17	17	17
2	17	17	13
3	14	14	9
4	12	10	7
5	5	5	5
6	5	5	5
7	5	5	5





5. Concorrência, mutex e semáforos

Quando dois ou mais processos acessam o mesmo recurso compartilhado sem coordenação, o resultado depende da ordem de intercalação: é uma **condição de corrida**. O trecho de código que manipula o recurso é a **seção crítica**, que deve ser executada por um processo de cada vez (**exclusão mútua**). O **mutex** é uma trava binária (adquirir/liberar); o **semáforo** é um contador com operações atômicas *wait* e *signal*, e um semáforo binário equivale a um mutex.

No KernelLab, cada dispositivo possui uma **fila FIFO de solicitações** protegida por um mutex (classe FifoMutex): quem chega entra na fila e só executa quando o dispositivo está livre e é o primeiro da fila, o que garante exclusão mútua e atendimento por ordem de chegada (sem inanição na fila). O mesmo cenário foi executado com e sem o mutex, com quatro threads (P3, P5, P6 e P4) concorrendo pelo disco:

Execução	Máx. acessos simultâneos	Violações de exclusão mútua	Ordem de atendimento
Com mutex + fila FIFO	1	0	P3 -> P5 -> P5 -> P6
Sem mutex	3	3	intercalada / imprevisível

Os valores da execução sem mutex dependem do escalonamento das threads pelo SO hospedeiro; o que é determinístico é que, com o mutex, o máximo de acessos simultâneos é sempre 1.

O acesso **exclusivo** (sessão) permite que um processo reserve o dispositivo para várias operações seguidas; no cenário, o backup (P5) executa duas gravações consecutivas sem que P6 intercale uma leitura no meio.

6. Segurança no acesso aos dispositivos

A proteção usa uma **matriz de controle de acesso**: linhas são processos (sujeitos), colunas são dispositivos (objetos) e cada célula lista as permissões (ler, gravar, executar, usar, exclusivo). Toda

requisição é verificada *antes* de entrar na fila: se o processo não possui a permissão, o acesso é negado e ele não ocupa nem bloqueia o dispositivo. A matriz do projeto segue o **princípio do menor privilégio**: cada processo recebe apenas os dispositivos declarados no enunciado. Foram decisões de projeto (não vindas do enunciado) a distribuição das permissões específicas (por exemplo, exclusivo no disco para o backup, e no áudio e rede para o alarme), e o dispositivo 'log', aberto à gravação de todos os processos. No cenário, P4 (alarme) tenta usar o disco, que não consta em sua lista, e é negado.

Proc	disco	rede	audio	log
P1	-	L G U	-	L G U
P2	-	L G U	-	G U
P3	L G U	-	-	G U
P4	-	G U E	G U E	L G U
P5	L G U E	-	-	G U
P6	L G X U	L U	-	G U

L = ler, G = gravar, X = executar, U = usar, E = acesso exclusivo.

7. Relação entre os conceitos e as decisões implementadas

Conceito	Decisão no KernelLab	Por quê (cenário MedControl)
Escalonamento por prioridade	Prioridade como política recomendada; preempção como variante	O alarme (prioridade 0) não pode esperar um monitoramento terminar
Efeito comboio / starvation	Flags de atraso excessivo e risco de starvation por algoritmo	Detectar quem é prejudicado (P4 no FCFS, P5 em SJF/Prioridade)
Justiça	Índice de Jain	Quantificar o que o Round Robin ganha em equilíbrio e perde em espera
Dados fora da lógica	JSON de processos, quantum, frames e sequência configuráveis	Reavaliar o servidor com cargas diferentes sem alterar código
Memória virtual	FIFO, LRU e Optimal com varredura de frames	Medir quanto uma política melhor ou mais memória reduzem faltas
Exclusão mútua	Mutex com fila FIFO por dispositivo	Impedir que backup, relatório e atualização gravem ao mesmo tempo no disco
Proteção	Matriz de acesso verificada antes da fila	Impedir que processos não autorizados usem recursos críticos (áudio do alarme)

8. Recomendação final e limitações

Política recomendada: escalonamento por prioridade com preempção, para a CPU; LRU (ou aproximação) para a substituição de páginas, por reduzir as faltas de 12 para 10 nesta sequência, enquanto o Optimal permanece só como referência; e acesso aos dispositivos por fila FIFO com mutex, mediado pela matriz de controle de acesso.

Limitações que permanecem:

- Risco de starvation do backup (P5): espera 24 unidades e é ultrapassado. A prioridade pura não resolve; um mecanismo de *envelhecimento* (aumentar gradualmente a prioridade de quem espera) seria a evolução natural, mas **não foi implementado**.
- A preempção só resolve o alarme porque ele é o processo mais prioritário; com vários alarmes simultâneos haveria disputa entre eles.
- O modelo ignora o custo (em tempo) da troca de contexto e as operações de E/S dos processos durante a execução na CPU: o bloqueio por dispositivo (Etapa 4) é simulado separadamente do

escalonamento (Etapa 1).

- Memória: a sequência de páginas é única e global; o simulador não relaciona os MB de cada processo com a quantidade de páginas, nem simula pré-carregamento ou tabela de páginas por processo.
- O SJF supõe burst conhecido; em um servidor real ele precisaria ser estimado.
- A matriz de acesso é estática e foi definida pelo projeto; não há autenticação nem revogação em tempo de execução.
- Os resultados valem para o conjunto de seis processos do enunciado; outras cargas podem mudar o ranking (o simulador permite testar).

9. Referências

- TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. (livro-base indicado na disciplina).
- UNIVASF. *Organização e Arquitetura de Computadores II - Sistemas Multiprogramados*. https://www.docentes.univasf.edu.br/max.santana/material/OAC-II/sistemas_multiprogramados.pdf
- PUC-Rio. *INF 1316 / INF 1019: Sistemas Operacionais - Cap. 1*. https://www-di.inf.puc-rio.br/~endler/courses/inf1019/transp/aulas-teoricas/cap-1.pdf
- TECNOBLOG. *O que é um sistema operacional?* https://tecnoblog.net/responde/o-que-e-um-sistema-operacional/
- JAIN, R.; CHIU, D.; HAWE, W. *A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems*. DEC Technical Report 301, 1984 (origem do índice de justiça usado).
- Unidades, videoaulas e textos-base da disciplina Sistemas Operacionais (AVA).

10. Links da entrega

Repositório GitHub: https://github.com/hugofelipe22/KernelLab

Vídeo pitch: https://drive.google.com/file/d/1LoJspIvDFljWNC1lrjUZcHG6iQ36kUoS/view?usp=sharing